Tanveer

LABORATORY REPORT: JAVA DATABASE CONNECTIVITY (JDBC)

1. Objective & Requirements

This laboratory report outlines the comprehensive step-by-step procedure, source code architecture, and database execution results for core Java Database Connectivity (JDBC) operations. The tasks include executing parameterised inputs using PreparedStatement, calling stored procedures via CallableStatement, and building a fully integrated lifecycle engine for Create, Read, Update, and Delete (CRUD) actions.

Task 1: PreparedStatement Data Entry & Table Display

Objective: Create a table 'Person' with specific department columns, insert real-world user data dynamically via parameterised structures, and fetch all entries inside an aligned matrix.

Database Schema Generation (SQL)

```
CREATE TABLE Person (
    Department INT,
    Department_name_Namevarchar VARCHAR(255),
    City VARCHAR(255)
);
```

Java Source Code (PreparedStatementDemo.java)

```
import java.sql.*;
import java.util.Scanner;

public class PreparedStatementDemo {
    private static final String URL = "jdbc:mysql://localhost:3306/lab_db";
    private static final String USER = "root";
    private static final String PWD = "password";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try (Connection conn = DriverManager.getConnection(URL, USER, PWD)) {
            String insertSQL = "INSERT INTO Person VALUES (?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
                System.out.print("Enter Department ID: ");
                pstmt.setInt(1, sc.nextInt()); sc.nextLine();
                System.out.print("Enter Department Name: ");
                pstmt.setString(2, sc.nextLine());
                System.out.print("Enter City: ");
```

```
                pstmt.setString(3, sc.nextLine());
                pstmt.executeUpdate();
                System.out.println(">> SUCCESS: Record inserted cleanly.");
            }
            System.out.println("\n--- Person Table Matrix Grid ---");
            try (Statement stmt = conn.createStatement();
                 ResultSet rs = stmt.executeQuery("SELECT * FROM Person")) {
                System.out.println("+------------+----------------------+------
------+");
                System.out.println("| Department | Department Name      | City
|");
                System.out.println("+------------+----------------------+------
------+");
                while(rs.next()) {
                    System.out.printf("| %-10d | %-20s | %-10s |\n",
                        rs.getInt(1), rs.getString(2), rs.getString(3));
                }
                System.out.println("+------------+----------------------+------
------+");
            }
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Execution Terminal Screenshot Simulation

```
[CONSOLE OUTPUT - PREPAREDSTATEMENTDEMO]
Connecting to database... Connected Successfully!
Enter Department ID: 101
Enter Department Name: Human Resources
Enter City: Hyderabad
>> SUCCESS: Record inserted cleanly.

--- Person Table Matrix Grid ---
+------------+----------------------+------------+
| Department | Department Name      | City       |
+------------+----------------------+------------+
| 101        | Human Resources      | Hyderabad  |
+------------+----------------------+------------+
```

3. Task 2: CallableStatement Row Deletion via Stored Procedure

Objective: Encapsulate database execution logic inside a native MySQL stored procedure and invoke it seamlessly inside the Java program via a CallableStatement interface container.

Stored Procedure Schema Definition (SQL)

```
DELIMITER //
CREATE PROCEDURE DeleteDepartment(IN deptId INT)
BEGIN
    DELETE FROM Person WHERE Department = deptId;
END //
DELIMITER ;
```

Java Source Code (CallableStatementDemo.java)

```
import java.sql.*;
import java.util.Scanner;

public class CallableStatementDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String url = "jdbc:mysql://localhost:3306/lab_db";
        System.out.print("Enter Department ID to Delete: ");
        int targetId = sc.nextInt();

        try (Connection conn = DriverManager.getConnection(url, "root",
"password");
             CallableStatement cstmt = conn.prepareCall("{call
DeleteDepartment(?)}")) {

            cstmt.setInt(1, targetId);
            int affected = cstmt.executeUpdate();
            System.out.println(">> Procedure executed safely.");
            System.out.println(">> Database Confirmation: Rows cleared: " +
affected);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Execution Terminal Screenshot Simulation

```
[CONSOLE OUTPUT - CALLABLESTATEMENTDEMO]
Establishing active link to relational instance...
Enter Department ID to Delete: 101
>> Procedure executed safely.
>> Database Confirmation: Rows cleared: 1
```

4. Task 3: Comprehensive Book Table CRUD Architecture

Objective: Design a structural framework tracking library book metrics (Serial Number, Book Number, Book Name, Price) executing sequentially Create, Read, Update, and Delete actions inside a single workflow transaction routine.

Java Source Code (CRUDOperationsDemo.java)

```
import java.sql.*;

public class CRUDOperationsDemo {
    private static final String URL = "jdbc:mysql://localhost:3306/lab_db";
    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, "root", "password"))
{
            Statement st = conn.createStatement();
            st.execute("CREATE TABLE IF NOT EXISTS Book (Serial_Number INT
AUTO_INCREMENT PRIMARY KEY, Book_Number INT, Book_name VARCHAR(255), Price DOUBLE)");

            // 1. CREATE
            PreparedStatement ins = conn.prepareStatement("INSERT INTO Book
(Book_Number, Book_name, Price) VALUES (?, ?, ?)");
            ins.setInt(1, 9005);
            ins.setString(2, "Mastering JDBC");
            ins.setDouble(3, 499.50);
            ins.executeUpdate();
            System.out.println("[CREATE] Row elements populated smoothly.");

            // 2. READ
            printGrid(conn);

            // 3. UPDATE
            PreparedStatement upd = conn.prepareStatement("UPDATE Book SET Price = ?
WHERE Book_Number = ?");
            upd.setDouble(1, 550.00);
            upd.setInt(2, 9005);
            upd.executeUpdate();
            System.out.println("[UPDATE] Modified target metrics successfully.");

            // 4. DELETE
            st.executeUpdate("DELETE FROM Book WHERE Book_Number = 9005");
            System.out.println("[DELETE] Dropped record index entries safely.");
        } catch (Exception e) { e.printStackTrace(); }
    }

    private static void printGrid(Connection conn) throws Exception {
        System.out.println("\n--- Current Book Database View ---");
        try (Statement s = conn.createStatement(); ResultSet r =
s.executeQuery("SELECT * FROM Book")) {
            while(r.next()) {
```

```
                System.out.printf("Serial: %d | BookNo: %d | Name: %s | Price:
%.2f\n",
                    r.getInt(1), r.getInt(2), r.getString(3), r.getDouble(4));
            }
        }
    }
}
```

Execution Terminal Screenshot Simulation

```
[CONSOLE OUTPUT - CRUDOPERATIONSDEMO]
Initialising relational table configurations...
[CREATE] Row elements populated smoothly.

--- Current Book Database View ---
Serial: 1 | BookNo: 9005 | Name: Mastering JDBC | Price: 499.50

[UPDATE] Modified target metrics successfully.
[DELETE] Dropped record index entries safely.
Workflow Execution Framework Complete.
```